Intercepting And Decoding Apple Push Notification Traffic From An iPhone

D.J. Leith
Trinity College Dublin
Dublin, Ireland
doug.leith@tcd.ie
17 October 2026

Abstract

Every iPhone keeps a persistent connection to the Apple Push Notification service (APNs). We describe how to intercept and decode APNs traffic alongside ordinary HTTPS traffic using `mitmdump` and a jailbroken iPhone 8 (iOS 16.7.11), give the message format used on the connection, and report what the messages contain.

1 Introduction

Measurement studies of the data that a phone sends often decrypt its HTTPS traffic using a man-in-the-middle proxy, e.g. [3, 4, 7–9, 11, 18, 19]. On an iPhone this usually misses one long-lived connection: the connection that the `apsd` daemon keeps open to the Apple Push Notification service (APNs) so that notifications can be delivered to apps and system services. It is missed for two reasons. It uses TCP port 5223, so a setup that redirects only ports 80 and 443 to the proxy lets it pass unnoticed. If it is redirected to the proxy anyway, or falls back to port 443 because port 5223 is blocked, the connection fails: the server's reply to the TLS protocol negotiation is rejected by the TLS library that `mitmproxy` uses (Section 2.2), so the TLS handshake between the proxy and Apple never completes. The connection also carries Apple's own binary protocol, not HTTP. With some extra work it can be decrypted and decoded in the same capture as the handset's HTTPS traffic, and its messages show what the handset sends to, and receives from, APNs:

1. *Intercepting APNs.* A port redirect, an extra trusted root CA and a change to the ALPN offer that `mitmproxy` sends to the server let `mitmdump` relay and decrypt the connection, and a `mitmproxy` addon decodes the messages. On a jailbroken iPhone with SSL Kill Switch installed, `apsd` also needs a ten-line fix to the tweak: without it the handset has no working push connection at all.
2. *Identifiers sent.* Each Connect message carries the handset's push token and its device certificate. The push token is a 32-byte value that APNs uses to address the handset. The APNs server (`courier.push.apple.com`) assigns it, and the handset also sends it to other Apple services that need to reach it, e.g. as `x-push-token` and `ptkn` [6]. The push token is persistent: in our tests it did not change for at least seven months. The device certificate is the X.509 certificate that Apple issues to the handset at activation. The handset authenticates to APNs by sending this certificate together with a nonce that it signs with the matching private key, which is stored in its keychain. We did not observe any hardware identifier such as the UDID or serial number, or any Apple account identifier, in these messages.
3. *Public IP address.* The server's ConnectResponse echoes the handset's public IPv4 address and a region code (`IE`).
4. *Push topics.* As the service requires, the handset tells the server which push topics it wants notifications for, as SHA-1 hashes of the topic names. These include the bundle identifiers of the third-party apps that use notifications.

Section 2 describes the interception setup, Section 3 the message format and Section 4 the content of the captured messages.

2 Interception Setup

2.1 Test Handset And Network

We use an Apple iPhone 8 (`iPhone10,1`, iOS 16.7.11, build 20H360), jailbroken with palera1n [15] in rootless mode, with the Sileo package manager and the ElleKit tweak injector. The handset connects to a Wi-Fi access point that runs `mitmdump` [10] in transparent mode on a Raspberry Pi. iptables rules redirect the handset's TCP traffic on ports 80, 443 and 5223 to the proxy, and the `mitmproxy` CA certificate is installed as trusted on the handset. Certificate pinning in system processes is bypassed with a modified version of SSL Kill Switch 3 [13][1]. We use Frida [17] to inspect `apsd` on the running handset.

2.2 The Connection, TLS And ALPN

Before connecting to APNs, `apsd` fetches a configuration "bag" over ordinary HTTPS from `init.push.apple.com`. This names the APNs servers: `courier.push.apple.com`, with 50 numbered host names `1-courier.push.apple.com` to `50-courier.push.apple.com` (the "couriers"). `apsd` then opens a TLS connection to a courier on TCP port 5223, or on port 443 if port 5223 fails. Figure 1 shows this courier connection when it is relayed by `mitmdump`.

The courier connection uses TLS 1.3. Older versions of APNs authenticated the handset with a TLS client certificate [1]. A proxy in the middle then cannot open its own TLS connection to Apple unless it has the handset's private key, so intercepting the connection meant first extracting that key from the handset. This is no longer necessary. The server still asks for a client certificate in the TLS handshake, but `apsd` sends none. Instead, the handset proves its identity in its first APNs message, by sending its device certificate and a value signed with the certificate's private key (Section 4). This signed value is not tied to the TLS connection, so a proxy can forward the message unchanged over its own TLS connection to Apple, and Apple accepts it.

[1] Source and a prebuilt package: https://github.com/doug-leith/poor-privacy-practices-of-the-apple-app-store and https://github.com/doug-leith/cydia.

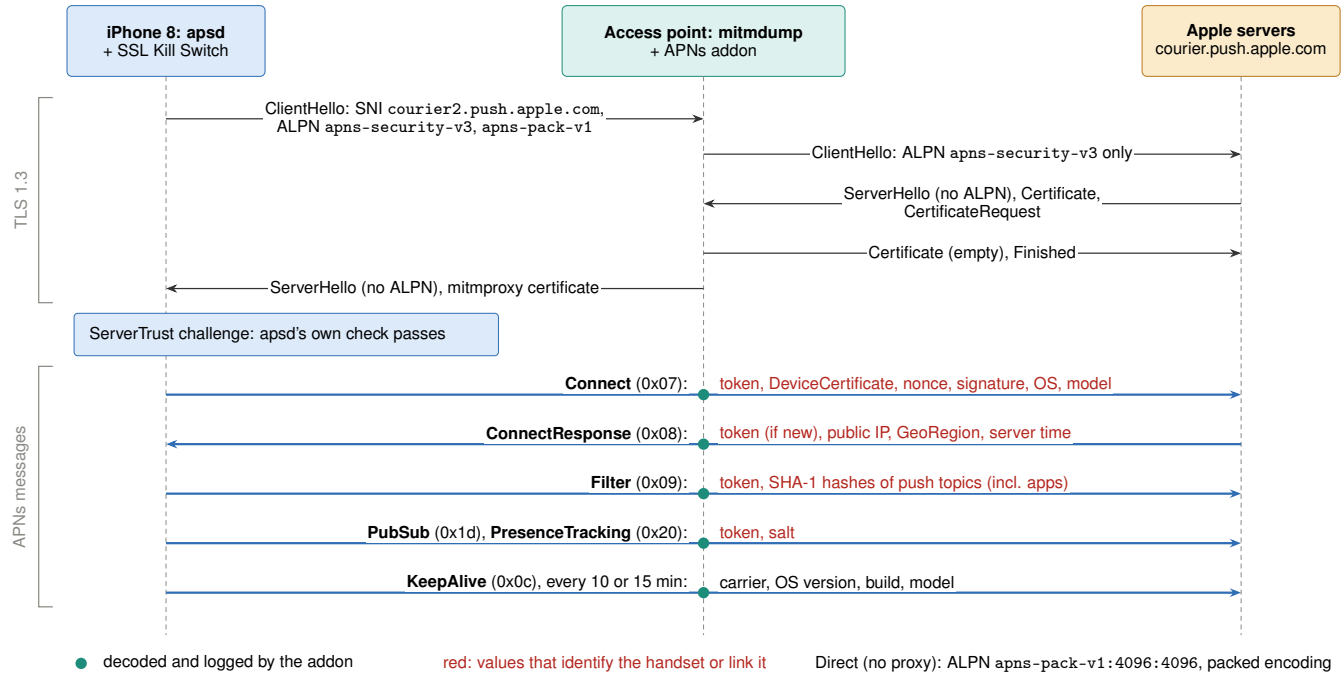


Figure 1: APNs connection from the iPhone through `mitmdump`. The proxy offers Apple only `apns-security-v3`; the server then selects no protocol, and both sides use the serial encoding. Red text marks values that identify the handset or link it to other traffic: the connect message carries the push token and the activation device certificate, the reply carries the handset's public IP address, and the filter message lists hashes of the push topics, including third-party apps.

In the TLS handshake the handset also lists the application protocols that it can speak, and the server picks one (TLS's ALPN extension). The handset lists `apns-security-v3` and `apns-pack-v1`, which correspond to two message encodings: a simple serial encoding (Section 3) and a compressed "packed" one (Appendix A). On a direct connection the server answers `apns-pack-v1:4096:4096`. This is not one of the values that the handset offered: the server has appended two numbers, which `apsd` uses as the sizes of the packed encoding's tables. Apple's TLS library on the handset accepts this answer, and `apsd` then uses the packed encoding. OpenSSL, which `mitmproxy` uses for its own connection to Apple, treats an answer that was not offered as an error (`BAD_EXTENSION`), and the handshake fails. This is why an unmodified `mitmproxy` breaks the connection.

Our `mitmproxy` addon works around this by offering Apple only `apns-security-v3`. The server does not pick that, but now the handshake succeeds with no protocol selected. `mitmproxy` passes this on to the handset, and `apsd` then uses the serial encoding. The two encodings use the same command and field numbers, and the messages that we recorded in each carried the same fields (Section 3).

`mitmproxy` must also accept the couriers' certificates. Some couriers presented a certificate issued under Apple Root CA, which is not in `mitmproxy`'s default list of trusted roots (from the certifi package), so it is necessary either to turn off `mitmdump`'s check of server certificates, with the option `--ssl-insecure`, or to add Apple Root CA to its trusted roots. We used both, in separate sessions. Other couriers presented a certificate from Apple's publicly trusted "Apple Public Server ECC CA 12 - G1", which needs neither.

2.3 SSL Kill Switch And apsd

SSL Kill Switch is a jailbreak tweak that makes iOS processes accept any server certificate, so that a proxy can decrypt their traffic. With it installed, `apsd` never works: it completes the TLS handshake but never sends its first APNs message, gives up after 30 seconds and tries again. `apsd`'s log on our handset showed ten failed attempts in a row. This happens with or without the proxy, so a handset with SSL Kill Switch installed has no working push notifications.

The reason is that `apsd` waits for a step that SSL Kill Switch removes. Normally, during the TLS handshake the system asks `apsd` to check the server's certificate (a "server trust challenge" sent to `-[APSTCPStream URLSession:task:didReceiveChallenge:completionHandler:]`), and `apsd` starts sending only once its check has passed. SSL Kill Switch disables certificate checking by replacing the system's verification routine (the BoringSSL callback set by `SSL_set_custom_verify`) with one that always succeeds, and that routine is also what sends the challenge. `apsd` is therefore never asked, and waits until it gives up. Our ten-line fix makes SSL Kill Switch leave the routine and the challenge alone in `apsd` only. `apsd` then runs its own check, which passes because SSL Kill Switch's other hooks (`SecTrustSetPolicies`, `SecTrustEvalua`

teWithError) make every certificate look valid,[2] and it connects normally, including through the proxy. With the fixed tweak, apsd connected through mitmdump at its first attempt after a restart, with no other instrumentation.

2.4 The mitmdump Addon

After the TLS handshake the courier connection carries APNs messages rather than HTTP, so mitmproxy records it as a raw TCP flow. Our addon picks out courier flows by their ALPN offer and server name, decodes the messages in each direction as they arrive, and logs them (Figure 2). It also writes them to a JSON Lines file and adds them to the flow's metadata, so they are saved in the .mitm capture file next to the raw decrypted bytes. The capture file stays a standard one that mitmproxy reads without the addon. However, mitmdump writes a connection to the capture file only when the connection closes or mitmdump exits normally, and a courier connection stays open for hours. The addon therefore also writes the raw decrypted bytes to the JSON Lines file as they arrive, so that the courier traffic is not lost if mitmdump stops abruptly. The same file, run as a script, decodes the courier flows from either file.

Besides the addon and the port 5223 redirect, mitmproxy needs two settings: --ssl-insecure (or the extra root certificate) described above, and a longer idle timeout. mitmproxy closes connections that have been idle for 600 seconds. The courier connection is idle between keep-alive messages, and on our handset the first keep-alive came 10 minutes after connecting, just after mitmproxy's timeout. With the default setting mitmproxy therefore closed the connection every 10 minutes, and apsd reconnected. With the timeout set to one hour the connection stayed open, so our addon raises mitmproxy's timeout to one hour unless it has been set explicitly.

2.5 Logging Inside apsd With Frida

We also used Frida to record the messages inside apsd itself, before they are encrypted and after they are decrypted, by hooking the method that queues outgoing messages and the method that receives incoming data. This shows the packed encoding, which the proxy never sees, and apsd's own interpretation of each incoming message, which we used to check our decoder (Section 3). Before we fixed SSL Kill Switch, we also used Frida to make the certificate check that the missing challenge would have triggered.

3 Protocol And Message Format

3.1 Commands And Fields

Every APNs message is a command: a one-byte command number followed by a list of fields, each with a one-byte field number. Table 1 lists the commands. Apple does not document the protocol, so we took the names of the commands and fields from apsd's own code. This is still laborious, partly manual work, but less so than reading all of the relevant code by hand.

For messages from the server, the field names come from apsd's message parser (-[APSProtocolParser _parseSerialMessage:...] and APNSPackDecoder), which stores each field of an incoming message under a name such as APSProtocolToken. Its code is not obfuscated and could be read directly, but it is a long set of nested branches, one for each command and field number. Instead of tracing them by hand, we ran the parser on test messages containing one field at a time, for every command number from 0 to 0x3f and field number from 1 to 40, and recorded the name under which it stored each field and the type that it converted the value to. This range covers every command and field number that the serial parser's code can match. Only two fields have no name in apsd: fields 5 and 12 of the ConnectResponse, which its parser discards.

Table 1: APNs commands. Direction: C, sent by the handset; S, sent by the server. The commands marked ∗ were observed in our captures.

No.	Command	Dir.
0x07	Connect*	C
0x08	ConnectResponse*	S
0x09	Filter (push topics)*	C
0x0a	PushNotification	C, S
0x0b	PushAck	C, S
0x0c, 0x0d	KeepAlive*, KeepAliveResponse*	C, S
0x0e	NoStorage	S
0x0f, 0x10	Flush, FlushResponse	C, S
0x11, 0x12	TokenGenerate, TokenGenerateResponse	C, S
0x13, 0x14	SetActiveInterval, SetActiveState*	C
0x15	TransportAck	C
0x16	KeepAliveAck*	S
0x17	Redirect	S
0x18, 0x19	TaskControl, TaskNotification	C, S
0x1b	MessageTracingAck	C
0x1d, 0x1e	PubSub*, PubSubUpdate*	C, S
0x20	PresenceTracking*	C, S

Messages from the handset are built by apsd methods such as -[APSProtocolParser copyConnectMessageWithToken:...], which take the values to send as arguments (the push token, the certificate, the nonce and so on) and write each one under a field number. We read from these methods which value goes under which field number, and named each field after the argument that it is filled from, e.g. Token or Nonce. For the Connect message we also checked this on a live connection: we logged with Frida the arguments that apsd passed to the builder, and found each of them under the expected field number in the message that the handset then sent.

The serial and packed encodings use the same command and field numbers: apsd's builders write each field into both encodings under the same number, and the messages that we recorded in each encoding carried the same fields. apsd's parser does recognise a few fields only in the packed encoding, e.g. three keep-alive intervals (fields 16 to 18) in the ConnectResponse.

3.2 Message Encoding

Through mitmdump, messages use the serial encoding (Figure 3). A message is the command number, the message length (4 bytes), and then the fields, each a field number (1 byte), a value length (2 bytes) and the value. Lengths are big-endian. Values are plain bytes with no type information: whether a value is a number, a string or binary data depends on the field. Table 2 lists the field numbers of the Connect message and the ConnectResponse, and Appendix B lists all of the field numbers that we found. On a direct connection

[2] Our Frida logs show that apsd never called its pinning check against Apple's push certificate authority (SecPolicyCreateApplePushService), although its configuration enables it. We have not established why.

```
-> Connect(0x07) Token=e30e...45f8 (32B), State=1, Flags=618, Interface=1, ActiveInterval=5, Carrier=WiFi, SoftwareVersion=16.7.11, SoftwareBuild=20H360,
↪  HardwareVersion=iPhone10,1, Certificate=3082...33fe (759B), Nonce=0000...d050 (17B), Signature=0101...0c2a (130B), ProtocolVersion=10, RedirectCount=0,
↪  TCPHandshakeTimeMs=2, DNSResolveTimeMs=28, TLSHandshakeTimeMs=147, ...
<- ConnectResponse(0x08) ConnectedResponse=0, MessageSize=5120, attr5=000a (2B), Capabilities=3, LargeMessageSize=51200, ServerTime=1792242105031, GeoRegion=IE,
↪  attr12=0000...3ac7 (8B), ClientIPAddress=c1...92 (4B)
-> Filter(0x09) Token=e30e...45f8 (32B), EnabledTopicHash=3e86...bde9 (20B), EnabledTopicHash=7429...423f (20B), ... (92 topic hashes)
-> PubSub(0x1d) PubSubMessageID=2, ChannelListPayload=0801...1801 (997B), BaseToken=e30e...45f8 (32B)
-> PresenceTracking(0x20) MessageID=10695208, PushToken=e30e...45f8 (32B), Salt=61...52, TrackingFlag=0, Timestamp=1792242105042
-> KeepAlive(0x0c) Carrier=WiFi, SoftwareVersion=16.7.11, SoftwareBuild=20H360, HardwareVersion=iPhone10,1, KeepAliveInterval=10, DelayedResponseInterval=600, State=4,
↪  SubState=4
<- PubSub(0x1d) PubSubMessageID=2, PubSubToken=e30e...45f8 (32B), PubSubStatus=0
<- PresenceTracking(0x20) PresenceActivityMessageID=10695208, PresenceActivityPushToken=e30e...45f8 (32B), PresenceActivityStatus=0
<- KeepAliveAck(0x16)
```

Figure 2: Start of a session as logged by the `mitmproxy` addon (-> handset to server, <- server to handset). Byte values are shortened to their first and last bytes. Fields 5 and 12 of the ConnectResponse have no name in `apsd`, which discards them, so the addon shows them by number (`attr5`, `attr12`).



Figure 3: Serial encoding of an APNs message. B, bytes; BE, big-endian.

the handset and the server use a second, compressed encoding, the packed encoding, which only matters for traffic recorded inside `apsd`, see Appendix A.

3.3 Verification

We compared the output of our decoder with `apsd`'s own interpretation of the same incoming messages, recorded with Frida. They agreed on every field that `apsd` reported: 18 fields in 7 messages on a direct connection (packed encoding) and 15 fields in 8 messages through `mitmdump` (serial encoding). Our decoder also shows two fields of the ConnectResponse, fields 5 and 12, that `apsd`'s parser discards. Field 5 was 10 in every session, and field 12 always had the same value as field 10, the time stamp in milliseconds that `apsd` stores as the server's time (`ServerTime`).

4 What The Messages Contain

4.1 Captures

A session is one courier connection, from the Connect message until the connection closes. On 17 October 2026 we recorded one direct session with Frida (packed encoding) and eight sessions through `mitmdump` (serial encoding). The couriers were in Apple's 17.57.146.0/24 address range, and the handset used the TLS server name `courier2.push.apple.com`. Each session starts with the same messages (Figure 2): Connect, ConnectResponse, Filter, PubSub, PresenceTracking and KeepAlive, and the server's replies. No push notification arrived during our sessions.

4.2 Connect And ConnectResponse

Table 2 lists the fields of the Connect message and of the ConnectResponse. The Connect message carries the push token, the device certificate, a nonce and a signature, the handset's iOS version, build and hardware model, connection timings and the reasons for earlier disconnections. We did not observe in it any hardware identifier such as the UDID, serial number or IMEI, or any Apple account identifier.

Table 2: Fields of the Connect message and of the ConnectResponse, with the values in a session through `mitmdump`. The push token, device certificate and client IP address are identifiers; shortened values end in . . . .

No.	Field	Value
Connect (0x07), handset to server		
1	Token	`e30e...45f8` (32 B)
2	State	1
5	Flags	0x26a
6	Interface	1
7	ActiveInterval	5
8 to 11	Carrier, OS version, build, hardware	`WiFi`, 16.7.11, 20H360, `iPhone10,1`
12	Certificate	activation device certificate (759 B)
13	Nonce	`00` + time (ms) + 8 random bytes
14	Signature	`01 01` + RSA-1024 signature of the nonce
16, 17	ProtocolVersion, RedirectCount	10, 0
19, 20, 25	DNS, TLS, TCP set-up time (ms)	e.g. 28, 147, 2
21	Failure reasons	reason and count
22, 26	LastConnected, DisconnectReason	time (ms), code
ConnectResponse (0x08), server to handset		
1	ConnectedResponse	0 (success)
3	Token	only when newly assigned
4, 8	MessageSize, LargeMessageSize	5120, 51200
6	Capabilities	3
10	ServerTime	time (ms)
11	GeoRegion	`IE`
19	ClientIPAddress	public IPv4 address (`c1...92`)
5, 12	unnamed, discarded by apsd	10; same as field 10

The device certificate. The certificate in the Connect message is the one that Apple issued to the handset when it was activated (the `DeviceCertificate` in the handset's activation record; our report on iPhone identifiers [6] describes how the handset obtains it): a 1024-bit RSA certificate issued by "Apple iPhone Device CA", valid from 17 February 2026 (the activation date) to 17 February 2029, with a UUID as its common name. It is also the certificate that the handset sends in the `x-push-cert` header when it registers for iMessage and FaceTime with Apple's identity servers (`identity.ess.apple.com`, `profile.ess.apple.com`) [6]; the two are byte-identical. Like the push token, the certificate is therefore a long-lived

value, valid for three years, that Apple's servers receive both on the APNs connection and in other requests from the handset. The nonce is a zero byte, the time in milliseconds and eight random bytes. The signature is the two bytes `01 01` followed by an RSA signature (PKCS#1 v1.5, SHA-1) over the nonce, which we verified with the certificate's public key. The signature lets the server check that the handset holds the private key that belongs to the certificate. Because the nonce contains the current time, an old signature cannot be replayed if the server checks that time. The ConnectResponse has a field that apsd stores as `ServerTimeForBadNonce` (field 7, not seen in our sessions), which suggests that the server does check it and tells a handset with a wrong clock what the server's time is.

The push token. The server assigns the push token. In apsd, the method that handles the ConnectResponse (`-[APSCourier _handleConnectedMessage:onInterface:]`) reads the token from it and, when one is present, stores it as the handset's push token. On later connections the handset sends the stored token in its Connect message and the server does not send it again, which is what we saw in all our sessions. We did not see a first assignment, since that requires clearing apsd's identity on the handset. The token in the sessions reported here, recorded on 17 October 2026, is byte-identical to the push token that the same handset sent seven months earlier, in the HTTPS traffic of our earlier App Store and Location Services experiments (captures from 16 March and 3 June 2026) [5, 7]. There it appears in the `x-push-token` header (base64) sent to Apple's identity servers and as the `ptkn` value (upper-case hexadecimal) sent to the Apple ID servers (`gsa.apple.com`, `gsas.apple.com`). Our report on iPhone identifiers [6] analyses these requests: of the 57 requests that carry the push token, 54 also carry the handset's UDID, 53 its serial number and 51 the Apple account identifier (ADSID). Apple's servers therefore receive the push token together with the handset's hardware identifiers and Apple account, and so are able to link the APNs connection to them. We cannot observe whether Apple does so.

The public IP address. The ConnectResponse echoes the handset's public IPv4 address and a region code, `IE`, which we take to be the server's geolocation of that address.

4.3 Filter: Push Topics

A push topic names a service or app that can receive notifications, usually by its bundle identifier, e.g. `com.apple.passd` for Wallet. After the ConnectResponse, the handset sends a Filter message listing the topics that it wants notifications for, each as the SHA-1 hash of the topic name, in lists that apsd calls enabled (74 topics), opportunistic (7), non-waking (7), ignored (4) and paused (empty). We identified 51 of the 92 hashes by hashing candidate names taken from the iOS system image, the handset's list of installed apps and its iMessage/FaceTime service definitions. Most are Apple services, e.g. `com.apple.mobileme.fmip` (Find My) and `com.apple.aa.idms` (Apple ID). The non-waking list includes the bundle identifiers of five apps: `com.apple.Maps`, `app.hallow.Hallow`, `com.dating.trans`, `com.dmfinity.glp1tracker` and `com.muslim.assist`. The four third-party apps were installed during our earlier experiments on this handset, which searched the App Store for dating, prayer and diabetes apps [7]. APNs needs the topics to route notifications, so this list necessarily tells Apple which push-enabled apps are installed. The Filter message carries the push token, and it is sent on the connection on which the handset has just presented its device certificate.

4.4 Other Messages

The PresenceTracking message, whose purpose we do not know, carries the push token, a message number, a time stamp and a 64-bit salt. The salt was the same in all sessions of one apsd process and changed when apsd restarted. The PubSub message subscribes four Apple services (`com.apple.mobileassetd`, `com.apple.sps.push`, `com.apple.symptomsd-diag`, `com.apple.triald`) to 33 channels, whose names (e.g. `customer ios`) suggest broadcast channels shared between devices. The KeepAlive message repeats the carrier, iOS version, build and hardware model. The server answers each KeepAlive with command 0x16, which carries no fields. apsd passes it to `-[APSCourier _handleKeepAliveAckMessage:onInterface:]`, which stops the timer that waits for a reply to the keep-alive.

5 Related Work

APNs protocol. PushProxy [1] was a man-in-the-middle proxy for the iOS 5 and OS X Lion version of APNs, from the time when the device certificate was presented as a TLS client certificate, and it documents the commands from Connect (0x07) to Flush (0x0f). pypush [2] implements the current serial protocol, including the connect, filter, keep-alive and message commands, the known topic names, and an interception proxy that, like our addon, offers only `apns-security-v3`. It gets apsd to accept the proxy's certificate by patching apsd's trust check with Frida, whereas our `mitmdump` setup needs no Frida, only the corrected SSL Kill Switch (Section 2.3). NowSecure [12] describes the move of the device certificate from the TLS layer into the connect message. The Apple Wiki [20] notes the `apns-pack-v1` encoding and its introduction in iOS 10. rustpush [14] added an implementation of the packed encoding in April 2026.

Agreement with our results. The serial field numbers in pypush agree with those that we recovered from apsd, and apsd's parser names several that pypush leaves unnamed, e.g. field 19 of the ConnectResponse, the client IP address. The server's ALPN value `apns-pack-v1:4096:4096` appears in the documentation of Zeek's TLS log [21], but we have not found a description of its effect on OpenSSL-based proxies, or of the effect of SSL Kill Switch on apsd.

6 Limitations

We used a single handset with a single iOS version, iOS 16.7.11. No push notification arrived during our captures, so the fields of the PushNotification command come from apsd's parser and not from observed messages. We did not observe a first assignment of a push token: the evidence that the server assigns it comes from apsd's code together with the reply format. 41 of the 92 topic hashes remain unresolved, so the list of third-party apps is a lower bound. Through the proxy the connection uses the serial and not the packed encoding, and the proxy's ALPN offer differs from the handset's, so a server could in principle tell that the connection is proxied.

7 Use Of AI

This report was drafted by Claude (Anthropic, Claude Opus 5.5), working in Claude Code, which also carried out the analysis reported here under the author's direction. The author reviewed the report.

References

[1] M. Frister. 2015. PushProxy: a man-in-the-middle proxy for iOS and OS X push notifications. https://github.com/mfrister/pushproxy. Accessed October 2026.

[2] JJTech. 2026. pypush: a Python library for Apple's internal APIs (APNs, IDS). https://github.com/JJTech0130/pypush. Accessed October 2026.

[3] Konrad Kollnig, Anastasia Shuba, Reuben Binns, Max Van Kleek, and Nigel Shadbolt. 2022. Are iPhones Really Better for Privacy? A Comparative Study of iOS and Android Apps. *Proceedings on Privacy Enhancing Technologies* 2022, 2 (2022), 6–24. https://doi.org/10.2478/popets-2022-0033

[4] D.J. Leith. 2026. Cookies, identifiers and other data that google silently stores on android handsets. *Computers & Security* 165 (2026), 104850. https://doi.org/10.1016/j.cose.2026.104850

[5] D.J. Leith. 2026. Decoding What iOS Location Services Send to Apple: An LLM Is an Unreliable Analyst but a Useful Toolmaker. Technical report, Trinity College Dublin.

[6] D.J. Leith. 2026. Device And User Identifiers In iPhone Network Traffic: Activation, Attestation And Anisette. Technical report, Trinity College Dublin. https://github.com/doug-leith/poor-privacy-practices-of-the-apple-app-store/blob/main/iphone_identifiers.pdf.

[7] D.J. Leith. 2026. Poor Privacy Practices Of The Apple App Store: Cookies, Advertising and Tracking Of Users. arXiv:2610.05475. https://arxiv.org/abs/2610.05475

[8] Douglas J. Leith. 2021. Mobile Handset Privacy: Measuring the Data iOS and Android Send to Apple and Google. In *Proc SecureComm, Security and Privacy in Communication Networks, LNICST 399*. 231–251. https://doi.org/10.1007/978-3-030-90022-9_12

[9] Haoyu Liu, Paul Patras, and Douglas J. Leith. 2023. On the data privacy practices of Android OEMs. *PLoS ONE* 18, 1 (2023), e0279942. https://doi.org/10.1371/journal.pone.0279942

[10] mitmproxy project. 2026. mitmproxy: a free and open source interactive HTTPS proxy. https://mitmproxy.org. Version 12.2.3.

[11] Trung Tin Nguyen, Michael Backes, Ninja Marnau, and Ben Stock. 2021. Share First, Ask Later (or Never?) Studying Violations of GDPR's Explicit Consent in Android Apps. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 3667–3684. https://www.usenix.org/conference/usenixsecurity21/presentation/nguyen

[12] NowSecure. 2021. Reverse Engineering iMessage: Leveraging the Hardware to Protect the Software. https://www.nowsecure.com/blog/2021/01/27/reverse-engineering-imessage-leveraging-the-hardware-to-protect-the-software/.

[13] NyaMisty. [n. d.]. SSL Kill Switch 3. https://github.com/NyaMisty/ssl-kill-switch3. Accessed October 2026.

[14] OpenBubbles. 2026. rustpush: Apple Push Notification service, IDS and iMessage client in Rust. https://github.com/OpenBubbles/rustpush. Packed APS encoding added 14 April 2026. Accessed October 2026.

[15] palera1n team. 2026. palera1n: iOS 15+ jailbreak for checkm8 devices. https://palera.in. Version 2.2.1.

[16] R. Peon and H. Ruellan. 2015. HPACK: Header Compression for HTTP/2. RFC 7541.

[17] O.A.V. Ravnås. 2026. Frida: dynamic instrumentation toolkit. https://frida.re. Version 17.18.0.

[18] Abbas Razaghpanah, Rishab Nithyanand, Narseo Vallina-Rodriguez, Srikanth Sundaresan, Mark Allman, Christian Kreibich, and Phillipa Gill. 2018. Apps, Trackers, Privacy, and Regulators: A Global Study of the Mobile Tracking Ecosystem. In *Proc NDSS*.

[19] Jingjing Ren, Martina Lindorfer, Daniel J Dubois, Ashwin Rao, David Choffnes, and Narseo Vallina-Rodriguez. 2018. Bug Fixes, Improvements,... and Privacy Leaks. In *Proc NDSS*.

[20] The Apple Wiki. 2026. Apple Push Notification Service. https://theapplewiki.com/wiki/Apple_Push_Notification_Service. Accessed October 2026.

[21] Zeek project. 2026. Zeek Log Formats: ssl.log. https://docs.zeek.org/en/master/reference/logs/ssl.html. Accessed October 2026.

A The Packed Encoding

On a direct connection, without the proxy, the handset and the server use the packed encoding instead of the serial one. It uses the same command and field numbers (Section 3.1) but fewer bytes. It saves space in two ways: numbers and lengths are written in as few bytes as possible, and a value that was sent earlier on the connection, such as the push token, can be replaced by a short reference to it, as in the HPACK header compression of HTTP/2 [16]. We worked out the format by having `apsd`'s own encoder (`APNSPackEncoder`) encode values that we chose and its decoder (`APNSPackDecoder`) decode byte strings that we chose, and we checked our decoder (in `apns_mitm.py`) against 800 random messages produced by `apsd`'s encoder.

Table 3: First byte of a packed value. L, length; V, value; S, sign; N, number of bytes minus one; C, number of entries; B, boolean. Lengths and counts use HPACK integer encoding with the prefix shown.

First byte	Value
`00LLLLLL`	bytes, 6-bit length prefix
`010LLLLL`	UTF-8 string, 5-bit length prefix
`100VVVVV`	unsigned integer 0 to 31
`1010SNNN`	integer, N+1 bytes big-endian magnitude
`1100CCCC`	dictionary of C entries
`1101CCCC`	array of C values
`1110000B`	boolean

Several lengths and counts use HPACK's integer encoding [16, Section 5.1]. A number smaller than $2^N - 1$ is stored in the low N bits of a byte, the prefix. A larger number sets those N bits to one, and the rest of the number follows in the next bytes, seven bits per byte, least significant first, with the top bit set on every byte except the last.

A message (Figure 4(a)) is the command number (1 byte), the length of the rest of the message (an HPACK integer with an 8-bit prefix) and the fields. Each field starts with a header byte (Figure 4(b)). Its top two bits, `MM`, say how the value is given: 00, the value follows; 01, the value follows and is also added to the value table; 10, an index into the value table follows instead, as an HPACK integer with an 8-bit prefix. The other six bits give the field number. If `K` and `E` are both 0, the field number is `IIII` (0 to 15). If `K` is 0 and `E` is 1, the field number is written out as a big-endian number in the bytes that follow; `IIII`, as an HPACK integer with a 4-bit prefix, gives the number of those bytes minus one, and the field number is added to the key table. If `K` is 1, the field number is the key-table entry whose index is in the low five bits.

A value starts with a byte that gives its type (Table 3). Byte strings and UTF-8 strings give their length in the low bits of that byte, followed by the bytes. Unsigned integers from 0 to 31 fit in the byte itself. Other integers give a sign bit `S` and their number of bytes, followed by the magnitude, big-endian. A dictionary gives its number of entries, followed by the entries; each entry is a header byte and a value, like a field, but its key is a UTF-8 string, written out (`E` = 1) or taken from the key table (`K` = 1). An array gives its number of values, followed by the values. Dictionaries and arrays carry structured values such as notification contents, which `apsd` converts back into Apple's binary property-list format.

Both tables are kept separately for each direction, and the most recent entry has index 0. The value table has a size limit, 4096 bytes on our connection (the numbers in the server's ALPN answer); each value counts as its length plus 33 bytes, and the oldest values

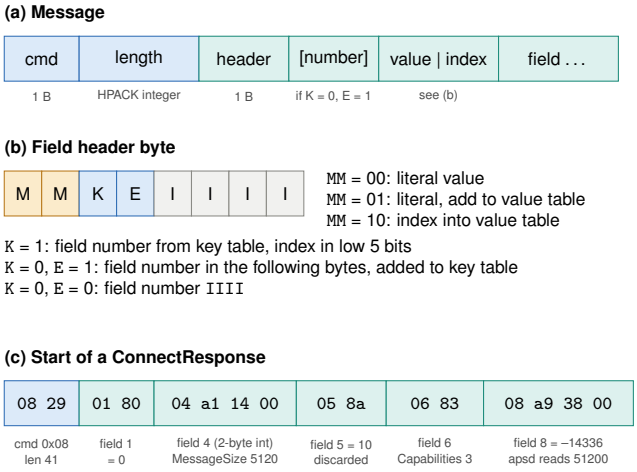


Figure 4: Packed encoding. (a) Message. (b) Header byte of a field. (c) The first 14 bytes of a ConnectResponse captured on a direct connection: small integers take a single byte, and the server writes the 16-bit message size 51200 as -14336 **(sign bit set, magnitude 0x3800).**

are dropped when the limit is reached. The key table holds 32 entries and is shared by field numbers and dictionary keys. Because references point back to earlier messages, a packed connection can only be decoded from its start.

Integers are written as a sign and a magnitude, not in two's complement, so the sender decides whether a value is negative. The server treats the 16-bit large message size 51200 (0xC800) as a signed number and sends it as −14336, with the sign bit set and the magnitude 0x3800 (Figure 4(c)). `apsd` stores the decoded −14336 in a 16-bit field, where it becomes 0xC800, i.e. 51200, again. To show the values that `apsd` uses, our decoder applies `apsd`'s field sizes in the same way.

B Field Numbers

Table 4 lists the field numbers of every command, for messages sent by the handset (C) and by the server (S). The names of fields sent by the server are those under which `apsd`'s parser stores them, without the prefix `APSProtocol`; the names of fields sent by the handset come from the values that `apsd`'s message builders write (Section 3.1). Fields 5 and 12 of the ConnectResponse, which `apsd` discards, are not listed. The Filter message has two field numbers for each of its five lists of topic hashes: `apsd` writes a hash of up to 31 bytes, such as the 20-byte SHA-1 hashes in our captures, under fields 2 to 6, and a longer hash under fields 7 to 11, which did not occur in our captures. The table is generated from our decoder's field tables.

Table 4: Field numbers and names of each command. Direction: C, sent by the handset; S, sent by the server.

No.	Command	Dir.	Field numbers and names
0x07	Connect	C	1 Token, 2 State, 5 Flags, 6 Interface, 7 ActiveInterval, 8 Carrier, 9 SoftwareVersion, 10 SoftwareBuild, 11 HardwareVersion, 12 Certificate, 13 Nonce, 14 Signature, 16 ProtocolVersion, 17 RedirectCount, 19 DNSResolveTimeMs, 20 TLSHandshakeTimeMs, 21 ConnectionFailureReason, 22 LastConnected, 25 TCPHandshakeTimeMs, 26 DisconnectReason
0x07	Connect	S	1 Token, 2 PresenceStatus
0x08	ConnectResponse	S	1 ConnectedResponse, 2 ServerMetadata, 3 Token, 4 MessageSize, 6 Capabilities, 7 ServerTimeForBadNonce, 8 LargeMessageSize, 9 RedirectReason, 10 ServerTime, 11 GeoRegion, 16 MinKeepAliveInterval, 17 ExpectedKeepAliveInterval, 18 MaxKeepAliveInterval, 19 ClientIPAddress
0x09	Filter	C	1 Token, 2 EnabledTopicHash, 3 IgnoredTopicHash, 4 OpportunisticTopicHash, 5 PausedTopicHash, 6 NonWakingTopicHash, 7 EnabledTopicHashLong, 8 IgnoredTopicHashLong, 9 OpportunisticTopicHashLong, 10 PausedTopicHashLong, 11 NonWakingTopicHashLong
0x0a	PushNotification	C	1 TopicHash, 2 Token, 3 Payload, 4 MessageID, 25 LastRTT
0x0a	PushNotification	S	1 Token, 2 TopicHash, 3 Payload, 4 MessageID, 5 MessageExpiry, 6 MessageTimestamp, 9 MessageStorageFlags, 13 MessagePriority, 15 BaseToken, 21 MessageTracingUUID, 24 MessageCorrelationIdentifier, 26 MessageAPNFlags, 28 MessagePushType, 29 MessagePubSubInfo
0x0b	PushAck	C	1 Token, 4 MessageID, 8 Status
0x0b	PushAck	S	6 AckTimestamp, 8 DeliveryStatus
0x0c	KeepAlive	C	1 Carrier, 2 SoftwareVersion, 3 SoftwareBuild, 4 HardwareVersion, 5 KeepAliveInterval, 6 DelayedResponseInterval, 8 UsingServerStats, 9 State, 10 SubState
0x0d	KeepAliveResponse	S	1 KeepAliveResponse
0x0e	NoStorage	S	1 Destination
0x0f	Flush	C	1 WantPaddingLength, 2 Padding
0x0f	Flush	S	1 FlushWantPadding
0x10	FlushResponse	C	2 Padding
0x11	TokenGenerate	C	1 BaseToken, 2 TopicHash, 3 AppId, 4 ExpirationTTL, 5 Type, 6 VapidPublicKeyHash
0x12	TokenGenerateResponse	S	1 AppTokenGenerateResponse, 2 AppTokenGenerateResponseToken, 3 AppTokenGenerateResponseTopicHash, 4 AppTokenGenerateResponseAppId
0x13	SetActiveInterval	C	1 ActiveInterval
0x14	SetActiveState	C	1 ActiveState, 2 ActiveInterval
0x17	Redirect	S	2 ServerMetadata, 9 RedirectReason
0x18	TaskControl	C	1 TaskMessageId, 2 TaskControlPayload
0x18	TaskControl	S	1 TaskMessageId, 2 TaskControlPayload, 3 TaskStatus
0x19	TaskNotification	S	1 TaskMessageId, 2 TaskNotificationPayload, 3 TaskStatus
0x1b	MessageTracingAck	C	1 TopicHash, 2 TracingUUID, 3 Status, 4 Token
0x1d	PubSub	C	1 PubSubMessageID, 2 ChannelListPayload, 3 BaseToken
0x1d	PubSub	S	1 PubSubMessageID, 2 PubSubPayload, 3 PubSubToken, 4 PubSubStatus, 5 PubSubTimeout
0x1e	PubSubUpdate	S	1 PubSubUpdateMessageID, 2 PubSubUpdatePayload, 3 PubSubUpdateToken, 4 PubSubUpdateTimestamp
0x20	PresenceTracking	C	1 MessageID, 2 PushToken, 3 Salt, 4 TrackingFlag, 5 Timestamp
0x20	PresenceTracking	S	1 PresenceActivityMessageID, 2 PresenceActivityPushToken, 3 PresenceActivityStatus